

# CODEJENN: A simple C++ neural network generator for physics applications

Jay Arcities<sup>a</sup>, Pavel Popov<sup>a</sup>, Eric J. Ching<sup>b</sup>, Kamal Viswanath<sup>b</sup>, Ryan F. Johnson<sup>b</sup>

<sup>a</sup>*San Diego State University, Aerospace Engineering Department, San Diego, CA 92182*

<sup>b</sup>*U.S. Naval Research Laboratory, Laboratories of Computational Physics and Fluid Dynamics, Washington, DC 20375*

---

## Abstract

Machine learning has shown speedups for numerical methods in physics applications, but integrating Python-based libraries into high-performance C++ solvers creates performance bottlenecks. We present CODEJENN, which bridges this gap by auto-generating self-contained C++ code from trained Keras models for inference. This eliminates external dependencies through minimal inlined functions, allowing seamless integration into existing frameworks. We describe the Keras-to-C++ workflow, supported architectures, and limitations. CODEJENN is demonstrated through inference benchmarks against Keras (eager and JIT modes) and a CFD test case modeling viscosity in a hydrogen-air mixing layer, showing speedups without sacrificing accuracy.

*Keywords:* Deep Learning, Computational Physics, Inference, Code Generation, Neural Networks

---

| Nr. | Code metadata description                                           | Please fill in this column                                                                                    |
|-----|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| C1  | Current code version                                                | v1.0                                                                                                          |
| C2  | Permanent GitHub link to code/repository used for this code version | <a href="https://github.com/jarcities/codejenn">https://github.com/jarcities/codejenn</a>                     |
| C3  | Legal Code License                                                  | NRL Open License Agreement                                                                                    |
| C4  | Code versioning system used                                         | Git                                                                                                           |
| C5  | Software code languages, tools, and services used                   | Python 3.12, C++23                                                                                            |
| C6  | Compilation requirements, operating environments & dependencies     | Keras 3, Anaconda Distribution                                                                                |
| C7  | If available Link to developer documentation/manual                 | <a href="https://github.com/jarcities/codejenn/readme.md">https://github.com/jarcities/codejenn/readme.md</a> |
| C8  | Support email for questions                                         | jarcities@sdsu.edu                                                                                            |

Table 1: Code metadata

## 1. Motivation and Significance

Machine learning has shown promise when integrated with numerical schemes for partial differential equations (PDEs), enabling research in data-driven preconditioners [1, 2], surrogate constitutive models [3], and physics-informed neural networks [4, 5, 6]. In computational fluid dynamics (CFD), neural networks are applied to accelerate simulations, turbulence modeling, and reduced order models [7]. Modern neural network architectures are accessible through high-level libraries like ONNX [8], TensorFlow [9], Keras [10], and PYTORCH [11]. However, speed and accuracy are critical during inference. Most physics solvers are written in compiled languages (Fortran, C++, etc.) for better resource control, while ML libraries are primarily Python-based. This disparity often adds computational overhead. Furthermore, integrating complex ML architectures into established solvers often requires significant implementation effort and experimentation.

The gap between ML libraries and compiled solvers often forces researchers to rely on framework-provided C++ APIs, which can be incompatible with established parallelization and vectorization codes. Delegating tensor operations to external APIs limits optimization and the efficient use of computing resources. Bati and Bryngelson [12] developed ROSENNA, a FORTRAN-based encoder that generates executable code for MLPs and RNNs from ONNX models. We developed CODEJENN, a C++-based encoder that generates standalone code for trained Keras models. CODEJENN produces all supporting activation and propagation functions in plain C++, allowing users to optimize source code directly without external dependencies like LAPACK or EIGEN.

CODEJENN is designed to be extensible and repeatable. If support for a particular architecture is not already available, the source code is structured to allow an LLM to generate only the required logic for that architecture. This provides a non-invasive approach to extending the framework while minimizing token consumption and reducing the effort required to experiment with new architectures.

The remainder of the paper is structured as follows: Section 2 covers neural network principles; Section 3 describes the CODEJENN architecture and compatibility; Section 4 demonstrates the software through inference speed benchmarks and a CFD viscosity modeling case using Wilke’s model [13]; and Section 5 discusses the broader impact of CODEJENN, including its extensibility, potential applications, and future development.

## 2. Machine Learning Background

A neural network is a surrogate model used to approximate complex mappings from training data. CODEJENN generates code for multi-layer perceptrons (MLPs) and convolutional neural networks (CNNs). In an MLP, each layer performs an affine transformation using weights  $\mathbf{w}$  and biases  $\mathbf{b}$ , followed by an activation function  $\sigma$  to introduce non-linearity, as shown in Eq. 1. A typical MLP schematic is shown in Fig. 1.

$$\mathbf{a} = \sigma(\mathbf{w}^T \mathbf{x} + \mathbf{b}) \quad (1)$$

CNNs extend this formulation using kernels to extract spatial features from multi-channel inputs:

$$Y_{c_{out},i,j} = \sum_{c_{in}} \sum_m \sum_n K_{c_{out},c_{in},m,n} X_{c_{in},i+m,j+n} + b_{c_{out}}, \quad (2)$$

Here,  $c_{out}$  and  $c_{in}$  are the input and output channel index of the filter,  $m$  and  $n$  are the spatial indices in the filter,  $i$  and  $j$  are the indices of the input data, and  $K$  is the filter.

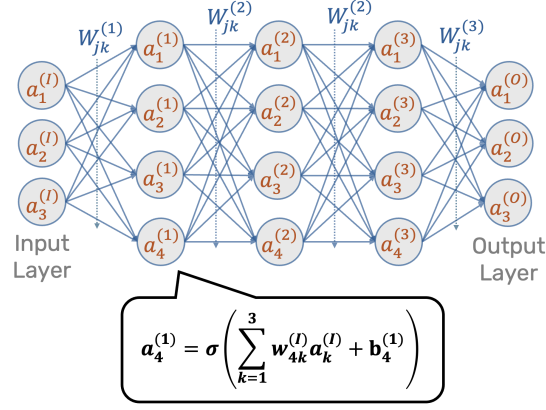


Figure 1: Schematic of a feedforward network (MLP), where  $a$  is the output from a given neuron,  $w_i^{jk}$  is the specific weight sent to that neuron given the edge. Superscript defines which layer, and subscript defines which neuron in that layer. The propagation step equation for  $a_4^{(1)}$  is shown in the message box.

In Eq. 2, the CNN layer output  $Y_{\text{out},i,j}$  is computed as a weighted combination of local input regions across all input channels, producing feature maps that capture local spatial patterns. A channel is the resulting map/matrix/image of a filter or pooling operation. A schematic of a CNN with one convolutional kernel (three filters) and two pooling layers is shown in Fig. 2. The structural differences highlighted when comparing Fig. 1 and Fig. 2 illustrate the diversity of machine-learned architectures and underlines their complexity in implementation in existing codebases.

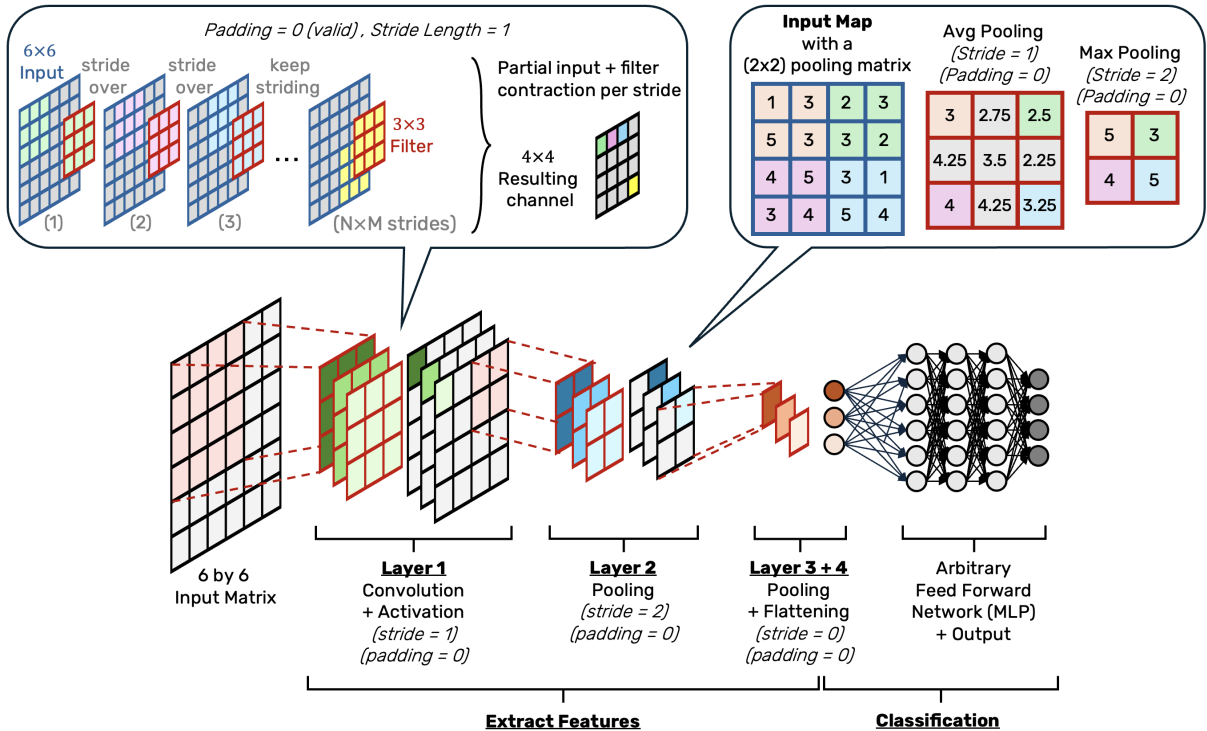


Figure 2: Schematic of a CNN with one convolution layer and two pooling layers. The kernels in layer one and two have three filters. The MLP after layer four is of any arbitrary shape and used for classification. The operation for applying a  $3 \times 3$  convolution kernel made of three filters is shown in the top left message box. The operation of applying a  $2 \times 2$  pooling matrix for both average and max pooling is shown in the top right message box. The resulting matrix of these operations is called a channel. Layer one would have three resulting channels.

### 3. Software Description

Figure 3 illustrates the workflow of CODEJENN. The process begins with the user obtaining data to train a neural network. That data is used to train a model using Keras, and then its parameters and hyperparameters are saved in a .keras or .h5 file. The user then utilizes that file through CODEJENN to generate self-contained C++ code that reproduces the trained neural network. The main principle of CodeJeNN is to inline as much as possible, minimize external dependencies, execute efficiently on local hardware, and remain fully self-contained using internal implementations.

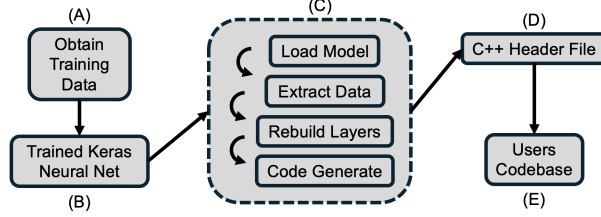


Figure 3: CODEJENN (C) code generates a KERAS-trained neural network into a C++ header file for the user’s computational physics solver (E).

#### 3.1. Keras Compatibility

Keras is chosen for its high-level API and backend integration with major frameworks (JAX, TensorFlow, PyTorch). It stores model architecture, hyperparameters, and parameters in a single file, allowing simplified model exchange and downstream CODEJENN code generation. Its emphasis on readability and accessibility is important for researchers who wish to train models and automatically generate equivalent C++ code without having to develop infrastructure in their codebase to allow for model inference.

#### 3.2. Code Generation

CODEJENN’s code generation proceeds in four steps: (1) load the saved model file using Keras’ `load_model` function, including any custom activation functions; (2) extract model parameters into a Python dictionary, where users can extend support for additional layers; (3) reconstruct the model from input to output layer; and (4) convert to a C++ header file.

Algorithm 1 presents a step-by-step procedure for how CODEJENN formats a generated MLP once the model is loaded in Python. First, the propagation step functions (similar to the neuron edges shown in Figure 1) are defined before the MLP function header. Then, within the MLP function body, there are three main sections. First, each layer’s parameters are generated. Second, the activation functions are defined as C++ lambda functions. Third, the propagation step function calls are produced, where inference is performed. CODEJENN generates neural networks in this format to enable downstream code optimization for users, where the propagation step functions benefit the most from modifications.

#### 3.3. Algorithm Design Structure

The generated code is written such that everything is known at compile time, allowing efficient cache utilization and fast inlined computation. As a result, an important design choice is the explicit use of syntax-level specifications, such as `void`, `inline`,

---

**Algorithm 1** Example of a code-generated MLP

---

```
1  Import packages: iostream, array, cmath, functional, etc.
2  Define propagation step functions:
3      Dense(outputs, inputs, weights, biases, input size, activation)
4      UnitNormalization(outputs, inputs, epsilon)
5  function MLP(input)
6      Print model parameters:
7          Layer weights and biases
8          Filter parameters
9          Input/Output normalization constants
10     Define lambda activation functions:
11         linear(input)
12         gelu(input)
13         custom_activation(input, index)
14     Propagation step:
15         Normalize  $x \leftarrow (x - \mu_{in})/\sigma_{in}$ 
16          $h_1 \leftarrow \text{Dense}(x, W_1, b_1, \text{linear})$ 
17          $h_2 \leftarrow \text{UnitNormalization}(h_1, \epsilon)$ 
18          $h_3 \leftarrow \text{gelu}(h_2)$ 
19          $h_4 \leftarrow \text{Dense}(h_3, W_4, b_4, \text{linear})$ 
20          $h_5 \leftarrow \text{custom\_activation}(h_4)$ 
21         Denormalize:  $y \leftarrow h_5 \cdot \sigma_{out} + \mu_{out}$ 
22     return  $y$ 
23 end function
```

---

`constexpr`, and `static`, which are applied consistently across arrays, headers, and function templates. Because minimizing computational cost is the top priority, CODEJENN aggressively inlines generated code wherever possible.

Most computational physics codes are already highly optimized, and further aggressive optimization of the generated code from CODEJENN prior to integration can be counterproductive. For example, nested parallelism arising from both the neural network and its evaluation can lead to thread oversubscription and degraded performance, where more threads are launched than available hardware resources [14]. As a result, compile-time optimizations and the use of external libraries are intentionally deferred to the user to explore. The objective is to achieve readable and accessible neural networks which can be manipulated even further once their implementation is satisfactory. This design choice also motivates placing propagation step functions at the top of the generated models (Algorithm 1), where they are most amenable to user-directed optimization. In addition, loops within lambda-based activation functions and lower-level function calls are structured to allow straightforward parallelization or vectorization when appropriate.

### 3.4. Capabilities and Limitations

CODEJENN supports the following activation functions

|           |                |            |               |
|-----------|----------------|------------|---------------|
| RELU [15] | LEAKYRELU [16] | SWISH [18] | SOFTMAX       |
| SIGMOID   | LINEAR         | SiLU [18]  | MISH [20]     |
| TANH      | ELU [17]       | GELU [19]  | SOFTPLUS [21] |

CODEJENN also supports custom activation functions by requiring users to (1) serialize the custom function when saving their model, and (2) define it in a Python script when loading. Examples are provided in the tutorials. Normalization parameters for inputs and outputs are also supported.

CODEJENN supports the following layers from the KERAS layer API [10]:

|                              |                                     |
|------------------------------|-------------------------------------|
| DENSE                        | UNIT NORMALIZATION                  |
| PURE ACTIVATION              | 1D, 2D, 3D CONVOLUTION              |
| DROPOUT                      | 1D, 2D, 3D TRANSPOSED CONV [23, 24] |
| 1D, 2D, 3D SPATIAL DROP [22] | 1D, 2D DEPTHWISE CONVOLUTION        |
| RESHAPE                      | 1D, 2D SEPARABLE CONVOLUTION        |
| FLATTEN                      | 1D, 2D, 3D MAX POOLING              |
| RESCALE                      | 1D, 2D, 3D AVERAGE POOLING          |
| LAYER NORMALIZATION          | 1D, 2D, 3D GLOBAL MAX POOLING       |
| BATCH NORMALIZATION          | 1D, 2D, 3D GLOBAL AVERAGE POOLING   |
| GROUP NORMALIZATION          |                                     |

There are limitations to CODEJENN. The primary limitation is that CODEJENN only supports single-branch neural networks. The model operates on a single input dataset and does not allow multiple independent datasets; however, this input dataset may be of arbitrary dimensionality. A secondary limitation is the number of configuration options within a layer that CODEJENN can support. For instance, in the Keras 3 API documentation [10], a 3D transposed convolution layer allows users to specify different regularizer options, which are not supported by CODEJENN. Because the code-generation pipeline supports only a subset of these configurations, users are directed to the source-code tutorials for full details.

To support development and integration, CODEJENN also includes a debugging API. KERAS has a `model.summary()` function for a trained and loaded model that provides a tabulated summary of the network architecture and layer dimensions. After code generation, CODEJENN compares what is extracted from that model against what `model.summary()` provides. Additionally, even when the C++ model generates successfully, there can still be logic/algebraic errors when performing inference. With the debug flag on, CODEJENN inserts print statements at the end of each layer, displaying the first ten output values of that layer. This allow pinpointing errors layer differences within a model. In practice, the majority of issues arise from user-defined custom activation functions, where the Python implementation of a custom activation does not match its C++ counterpart.

We have also considered variadic templates as an alternative to for loops. However, such templating would quickly increase compilation time, code size, and memory usage. For instance, consider an MLP with an input sample size of  $1 \times 400$  and a first hidden layer of 256 neurons. This configuration would require 102,400 weights in the first layer alone, making a fully templated implementation impractical. Consequently, this approach does not scale well to very large models due to the rapid growth in memory and compilation costs, which represents a significant limitation. For many physics applications, this limitation is acceptable because deployed models are intentionally kept small

and efficient to satisfy hardware and real-time constraints. CODEJENN’s aggressively inlined code generation is well suited to these use cases, where memory and computational costs must be considered across the entire numerical workflow and models are typically kept compact to conserve CPU time, memory, and computing-node resources. If a generated model exceeds the available hardware memory, computational costs can increase substantially, leading to inefficient execution. In practice, however, large models are often unnecessary for accurate inference in scientific computing applications. Instead, predictive performance is frequently influenced more by factors such as data preprocessing, sampling strategies, and feature representation than by increases in model size alone [25].

## 4. Illustrative Examples

We demonstrate the computational performance and implementation of CODEJENN via two test cases. First, we show the total wall-clock inference time of CODEJENN and Keras with an MLP and CNN. This is conducted on an Apple MacBook M1 Pro, with an M1 Pro chip containing 32 GB of memory and 10 cores (2 efficiency and 8 high-performance with peak performance of 3.22 GHz). Second, we apply a neural network implementation to replace the viscosity model in a simulation of a two-dimensional subsonic splitter-plate mixing layer. This is conducted on a compute cluster using a single node equipped with two AMD EPYC 7702 64-Core processors and 270 GB in memory.

### 4.1. Inference Speed

In our first test case, we compare the inference performance of both an MLP and CNN created in Keras and code-generated by CODEJENN. Keras requires a backend deep learning framework providing C++ API and tensor operations, such as TENSORFLOW, PYTORCH, or JAX. To ensure fair comparison, all models are tested on a single CPU core using a single thread. Since PYTORCH and TENSORFLOW are the most widely used frameworks, only these two backends are evaluated. By default, TENSORFLOW converts a model into graph mode by tracing hierarchical operations and executing them efficiently. PYTORCH defaults to eager execution. Keras also allows Just-in-Time (JIT) compilation, which compiles the computational graph into optimized C++ code. TensorFlow uses Accelerated Linear Algebra (XLA) compilation, whereas PyTorch utilizes TorchDynamo, AOTAutograd, and the Inductor compiler backend. These frameworks aggressively fuse operations, reduce memory usage, and apply target-specific optimizations. All Keras models are evaluated in eager mode with JIT compilation disabled to ensure fair baseline comparison. For completeness, we report JIT-compiled times to demonstrate performance achievable with aggressive backend optimization and provide a useful target for optimizing the generated C++ models.

Table 2 shows inference performance of CODEJENN compared to Keras for two 64-bit models, an MLP and CNN using 10,000 random data samples. Data samples are preloaded before benchmarking. The inference process performs a dry-run with a single sample to initialize system resources and cache, then measurements are taken over 10,000 iterations.

We use an MLP architecture with a 1,000-dimensional input passed through a 64-neuron dense layer with RELU, Batch Normalization, and 0.3 Dropout, followed by a 32-neuron RELU layer and a 100-neuron sigmoid output. The network is trained using Adam optimizer and binary cross-entropy loss on 10,000 randomly generated, standardized samples. Our CNN contained a  $10 \times 100 \times 1$  input, passed through convolutional

| Deep Learning Library  | MLP Inference (sec)               | CNN Inference (sec)               | Unbounded Memory CNN Inference (sec) |
|------------------------|-----------------------------------|-----------------------------------|--------------------------------------|
| Keras (TensorFlow)     | $12.01 \pm 0.24$                  | $19.14 \pm 0.29$                  | $21.36 \pm 0.38$                     |
| Keras (PyTorch)        | $18.20 \pm 0.19$                  | $26.77 \pm 1.13$                  | $27.53 \pm 0.48$                     |
| Keras (JIT-TensorFlow) | $1.21 \pm 0.03$                   | $2.17 \pm 0.18$                   | <b><math>3.60 \pm 0.05</math></b>    |
| Keras (JIT-PyTorch)    | $4.33 \pm 0.29$                   | $7.75 \pm 0.31$                   | $8.47 \pm 0.16$                      |
| CODEJENN               | <b><math>0.48 \pm 0.01</math></b> | <b><math>1.05 \pm 0.01</math></b> | $10.75 \pm 0.23$                     |

Table 2: MLP and CNN inference. The inference times represent the total wall-clock time to compute the inference of 10,000 random data samples sequentially for an MLP, CNN, and an unbounded memory CNN, created by Keras and CODEJENN. We test the Keras backend inference speed of PYTORCH and TENSORFLOW and their JIT compilation optimization. Each inference case is tested three times to ensure precision.

layers (8 filters,  $3 \times 3$ , softplus; 16 filters, tanh), a  $3 \times 3$  transposed convolution with RELU, flattened into a 32-neuron Mish layer, and 100-neuron softmax output. The CNN is trained with Adam optimizer and sparse categorical cross-entropy loss. To test CODEJENN capabilities, we ensured each layer utilized different tensor operations.

Inference in CODEJENN executes roughly 25 times faster than Keras using a TENSORFLOW backend and achieves nearly 38 times speedup with PYTORCH. When optimized with JIT compilation, CODEJENN remains roughly 2.5 times and 9 times faster than TENSORFLOW’s and PYTORCH’s JIT versions, respectively. For CNNs, speedup factors are smaller but substantial: approximately 18 times and 26 times relative to standard Keras for TENSORFLOW and PYTORCH, and 2 times and 7 times faster than their JIT-compiled counterparts.

The third column in Table 2 shows the results of evaluating a larger CNN architecture where the filters in the *Conv2DTranspose* layer are increased to 32 and the subsequent *Dense* layer is expanded to 64 neurons. The table shows that these architectural changes caused a significant increase in inference cost, as the number of floating-point operations required for those layers scaled quadratically. This expansion causes the model to transition from being compute-bound to memory-bound, as the larger weight matrices exceed the hardware’s L1 and L2 cache capacities and saturate available memory bandwidth. This results in high-latency stalls as the CPU waits for weight data from main memory. Although these memory limitations are inherent to larger neural network architectures, CODEJENN generates standalone, dependency-free C++ source code that users can modify and optimize directly for their target hardware.

#### 4.2. Inference Demonstration on a Splitter Plate

We embedded a neural network to replace viscosity in a hydrogen-air splitter-plate mixing layer simulation using OPENFOAM [26] and DETONATIONFOAM [27]. The demonstration targets mixture-averaged viscosity computed with Wilke’s mixing rule [13]. While viscosity evaluation is typically a minor cost in chemically reacting flow solvers, this case demonstrates that neural networks can be robustly embedded within a CFD workflow.

In the baseline case, the mixture dynamic viscosity is calculated from species-dependent viscosity fits and mole fractions as

$$\mu_{\text{mix}} = \sum_{i=1}^N \frac{X_i \mu_i}{\sum_{j=1}^N X_j \Phi_{ij}} \quad (3)$$



where  $X_i$  is the mole fraction of species  $i$ , and the interaction coefficient is

$$\Phi_{ij} = \frac{1}{\sqrt{8}} \left( 1 + \frac{M_i}{M_j} \right)^{-1/2} \left[ 1 + \left( \frac{\mu_i}{\mu_j} \right)^{1/2} \left( \frac{M_j}{M_i} \right)^{1/4} \right]^2, \quad (4)$$

with  $M_i$  the molar mass of species  $i$ . The species viscosity is evaluated using a log-polynomial fit in temperature:

$$\mu_i = 0.1 \exp(A_{1,i} + A_{2,i} \ln T + A_{3,i} (\ln T)^2 + A_{4,i} (\ln T)^3) \quad [\text{Pa} \cdot \text{s}]. \quad (5)$$

The domain is  $15 \text{ mm} \times 5 \text{ mm} \times 0.2 \text{ mm}$ . Two streams entered from the left and are separated at the centerplane at  $y = 2.5 \text{ mm}$ :

- **H<sub>2</sub> stream:**  $M = 0.3$ ,  $T = 350 \text{ K}$ ,  $p = 101,325 \text{ Pa}$ ,  $U \approx 427 \text{ m/s}$ .
- **Air stream:**  $M = 0.3$ ,  $T = 1500 \text{ K}$ ,  $p = 101,325 \text{ Pa}$ ,  $U \approx 226 \text{ m/s}$ .

The mesh consisted of two blocks per stream with  $190 \times 60 \times 1$  cells each. Streamwise grading ranged from  $20 \mu\text{m}$  at the inlet to  $200 \mu\text{m}$  at the outlet, with wall-normal grading symmetric about the centerplane ( $10 \mu\text{m}$  at the shear interface, coarsening to  $\approx 100 \mu\text{m}$  at boundaries). The top, bottom, and right boundaries are prescribed as zero-gradient supersonic outflow.

DETONATIONFOAM uses mixture-averaged transport employing the Wilke mixing rule. Chemistry is disabled to reduce computational cost. A quasi-steady flow field after five flow-through times ( $5 \times L/U_{x,\text{air}} \approx 3.32 \times 10^{-4} \text{ s}$ ) is shown in Figure 4, where unsteady mixing layer structures grow with downstream distance. We replaced Wilke's model with a neural network mapping thermochemical state directly to viscosity:  $(Y_{\text{H}_2}, Y_{\text{O}_2}, Y_{\text{N}_2}, T) \rightarrow \mu_{\text{mix}}$ .

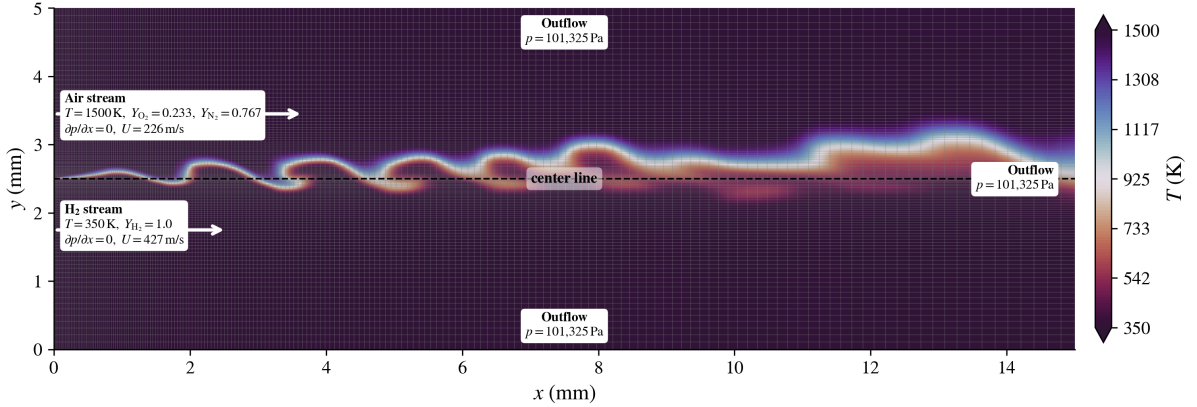


Figure 4: Temperature and flow domain

A fully-connected feed-forward neural network is generated using CodeJeNN and trained on  $10^4$  random states  $(Y_{\text{H}_2}, Y_{\text{O}_2}, Y_{\text{N}_2}, T)$  generated from a small OpenFOAM utility, `generateData`, contained on the CODEJENN repository that uses the `detonatFOAM` Wilke's rule (Eqs. (3)–(5)) as training target. The stratified distribution consisted of 60 % samples from a symmetric Dirichlet distribution with  $Y_i \sim \mathcal{U}(0, 1)$  and  $T \sim \mathcal{U}(200, 3000) \text{ K}$ , and 40 % along the physical mixing line,

$$\mathbf{Y}(\alpha) = \alpha \mathbf{Y}_{\text{fuel}} + (1 - \alpha) \mathbf{Y}_{\text{ox}}, \quad \alpha \sim \mathcal{U}(0, 1), \quad (6)$$

where  $\mathbf{Y}_{\text{fuel}} = (1, 0, 0)$  (pure  $\text{H}_2$ ) and  $\mathbf{Y}_{\text{ox}} = (0, 0.233, 0.767)$  (air), with temperature  $T(\alpha) = \alpha T_{\text{fuel}} + (1 - \alpha)T_{\text{ox}} + \delta T$ ,  $\delta T \sim \mathcal{U}(-200, 200)$  K,  $T_{\text{fuel}} = 350$  K, and  $T_{\text{ox}} = 1500$  K. This importance-sampling strategy ensures coverage of the pure-stream conditions that are underrepresented by the Dirichlet distribution yet dominate the inlet regions of the splitter-plate domain.

The network uses 3 hidden layers with 12 neurons each (385 parameters total) and Tanh activation. Normalized inputs and de-normalized output are embedded directly in C++ headers within a modified DETONATIONFOAM solver, requiring no external files at runtime. The network is verified against the Wilke model over 2 000 randomly sampled states. The relative error for each sample is computed as

$$\epsilon = \frac{|\mu_{\text{Wilke}} - \mu_{\text{NN}}|}{\mu_{\text{Wilke}}}, \quad (7)$$

yielding a mean error of 0.12 and a maximum error of 8.12.

Two simulations are run for five more flow-through times: one using the native Wilke model and another using the embedded neural network. Figure 5 presents the percent error for the  $x$ -direction velocity field, computed as  $100 \times |u_{x,\text{Wilke}} - u_{x,\text{NN}}| / u_{x,\text{Wilke}}$ . The neural network introduces minimal error, reaching a maximum of 0.25% only in the mixing layer. The absence of large-scale error structures elsewhere indicates that the neural-network viscosity model preserves overall flow evolution and instability development. These results demonstrate that CodeJeNN-generated networks can be embedded in CFD solvers as surrogates for constitutive models while maintaining close agreement with reference solutions.

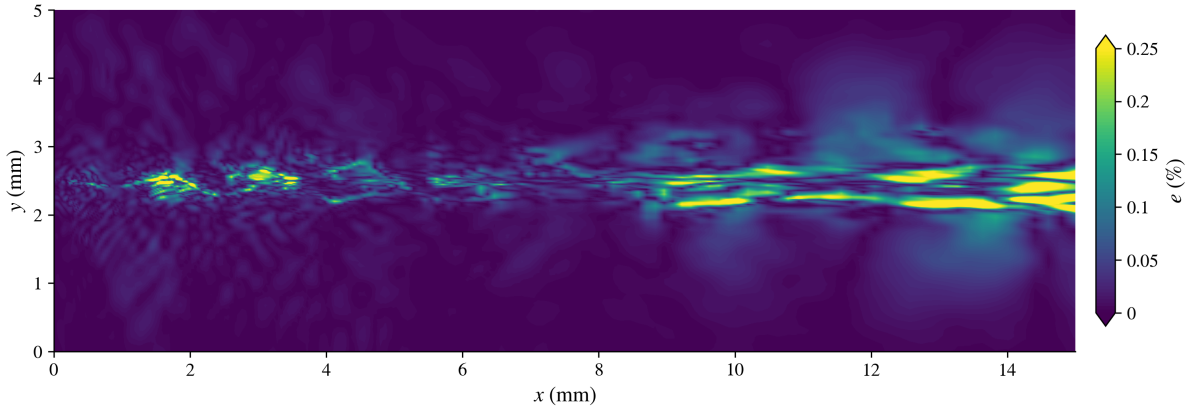


Figure 5: Error after one flow-through time

## 5. Impact

This software can allow researchers to integrate neural networks into their computational physics codebases with minimal additional infrastructure. Many physics experts may not wish to invest the time or resources to branch into machine learning solely to build code infrastructure. This product serves as a way to build inline infrastructure with only the prerequisite of a basic understanding of how neural networks perform inference. The key feature of CODEJENN is its ability to produce raw source code where everything is inlined and enclosed within a single header file. This negates the need to worry about exporting tensor operations from high-performance libraries, since many computational physics codes utilize all available hardware.

Integrating neural network inference into an established codebase using an LLM may require repeatedly supplying project-specific context and regenerating infrastructure whenever architectures change. Prior work has shown that software-engineering workflows can consume substantial token budgets, with context processing accounting for a large fraction of total token usage [28]. By automatically generating self-contained inference code, CodeJeNN reduces reliance on repeated LLM-assisted infrastructure generation and the associated token costs. Additionally, it has been shown that LLM reasoning often contains redundant tokens that increase inference costs without providing proportional gains in performance [29]. CODEJENN addresses this challenge by eliminating the need to repeatedly generate infrastructure code through an LLM, instead providing an easy-to-call function that computational physics practitioners can integrate directly while developing infrastructure on the fly.

Finally, CODEJENN is designed to be extensible and repeatable. If a particular architecture does not exist within CODEJENN, the way it is written allows a user to leverage an LLM to add the required logic with substantially fewer generated tokens. This enables a non-invasive workflow for generating new models while minimizing token consumption and reducing the effort required to experiment with new architectures. As a result, users can facilitate and implement neural network models within existing computational physics workflows in a shorter time period.

## 6. Conclusion

CODEJENN, a C++ neural network generation library, has been developed and demonstrated. This library is compatible with Keras, a commonly used deep learning API in computational physics, and enables users to implement neural networks in their codebases. The package does this by providing a self-contained C++ rendition of the Keras model. This allows the user to optimize raw source code without performance bottlenecks. We have demonstrated CODEJENN by replacing the viscosity model in a subsonic splitter-plate mixing layer simulation, along with evaluating its inference speed.

Future work will integrate SPEKTRAL [30], which is the GNN API for Keras that includes commonly used GNN layers such as message passing layers [31] and attention-based graphing layers [32]. Additionally, more activation functions will be added. We will also explore accelerating the inference performance of generated models by adding features to export functions using low-level high-performance libraries such as EIGEN [33]. This path may also include using different data structures for model hyperparameters, allowing users to homogenize the neural network within their codebase. CODEJENN is accessible at <https://github.com/jarcities/codejenn>.

## 7. Conflict of Interest

We wish to confirm that there are no known conflicts of interest associated with this publication and there has been no significant financial support for this work that could have influenced its outcome.

## Acknowledgements

The authors wish to acknowledge Dr. Eric Marineau and Dr. Jonathan Sosa of the Hypersonic Aerothermodynamics, High-Speed Propulsion and Materials Program of the Office of Naval Research Code 35 for directly supporting this work.

## Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author(s) used Claude from Anthropic in order to verify CFD test cases after infrastructure was already implemented. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

## References

- [1] P. Häusner, A. N. Juscafresa, J. Sjölund, Learning incomplete factorization preconditioners for GMRES (Dec. 2024). doi:10.48550/arXiv.2409.08262.  
URL <http://arxiv.org/abs/2409.08262>
- [2] Y. Li, P. Y. Chen, T. Du, W. Matusik, Learning Preconditioners for Conjugate Gradient PDE Solvers, in: A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, J. Scarlett (Eds.), Proceedings of the 40th International Conference on Machine Learning, Vol. 202 of Proceedings of Machine Learning Research, PMLR, 2023, pp. 19425–19439.  
URL <https://proceedings.mlr.press/v202/li23e.html>
- [3] J. N. Fuhg, G. Anantha Padmanabha, N. Bouklas, B. Bahmani, W. Sun, N. N. Vlassis, M. Flaschel, P. Carrara, L. De Lorenzis, A Review on Data-Driven Constitutive Laws for Solids, Archives of Computational Methods in Engineering 32 (3) (2025) 1841–1883. doi:10.1007/s11831-024-10196-2.  
URL <https://doi.org/10.1007/s11831-024-10196-2>
- [4] M. Raissi, P. Perdikaris, G. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, Journal of Computational Physics 378 (2019) 686–707. doi:10.1016/j.jcp.2018.10.045.  
URL <https://linkinghub.elsevier.com/retrieve/pii/S0021999118307125>
- [5] C. Zhao, F. Zhang, W. Lou, X. Wang, J. Yang, A comprehensive review of advances in physics-informed neural networks and their applications in complex fluid dynamics, Physics of Fluids 36 (10) (2024) 101301. doi:10.1063/5.0226562.  
URL <https://pubs.aip.org/pof/article/36/10/101301/3315125/A-comprehensive-review-of-advances-in-physics>
- [6] S. Cai, Z. Mao, Z. Wang, M. Yin, G. E. Karniadakis, Physics-informed neural networks (PINNs) for fluid mechanics: a review, Acta Mechanica Sinica 37 (12) (2021) 1727–1738. doi:10.1007/s10409-021-01148-1.  
URL <https://link.springer.com/10.1007/s10409-021-01148-1>
- [7] R. Vinuesa, S. L. Brunton, Enhancing computational fluid dynamics with machine learning, Nature Computational Science 2 (6) (2022) 358–366. doi:10.1038/s43588-022-00264-7.  
URL <https://doi.org/10.1038/s43588-022-00264-7>
- [8] J. Bai, F. Lu, K. Zhang, others, ONNX: Open Neural Network Exchange (2019).  
URL <https://github.com/onnx/onnx>

- [9] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Y. Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, Xiaoqiang Zheng, TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems (2015).  
URL <https://www.tensorflow.org/>
- [10] F. Chollet, others, Keras (2015).  
URL <https://keras.io>
- [11] J. Ansel, E. Yang, H. He, N. Gimselshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. Luk, B. Maher, Y. Pan, C. Puhersch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, M. Suo, P. Tillet, E. Wang, X. Wang, W. Wen, S. Zhang, X. Zhao, K. Zhou, R. Zou, A. Mathews, G. Chanan, P. Wu, S. Chintala, PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation, in: 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24), ACM, 2024. doi:10.1145/3620665.3640366.  
URL <https://docs.pytorch.org/assets/pytorch2-2.pdf>
- [12] A. Bati, S. H. Bryngelson, RoseNNA: A performant, portable library for neural network inference with application to computational fluid dynamics, Computer Physics Communications 296 (2024) 109052. doi:10.1016/j.cpc.2023.109052.  
URL <https://linkinghub.elsevier.com/retrieve/pii/S0010465523003971>
- [13] C. R. Wilke, A Viscosity Equation for Gas Mixtures, The Journal of Chemical Physics 18 (4) (1950) 517–519. doi:10.1063/1.1747673.  
URL <https://pubs.aip.org/jcp/article/18/4/517/314304/A-Viscosity-Equation-for-Gas-Mixtures>
- [14] Y. Yan, J. R. Hammond, C. Liao, A. E. Eichenberger, A Proposal to OpenMP for Addressing the CPU Oversubscription Challenge, in: N. Maruyama, B. R. De Supinski, M. Wahib (Eds.), OpenMP: Memory, Devices, and Tasks, Vol. 9903, Springer International Publishing, Cham, 2016, pp. 187–202. doi:10.1007/978-3-319-45550-1\_14.  
URL [http://link.springer.com/10.1007/978-3-319-45550-1\\_14](http://link.springer.com/10.1007/978-3-319-45550-1_14)
- [15] V. Nair, G. E. Hinton, Rectified linear units improve restricted boltzmann machines, in: Proceedings of the 27th International Conference on International Conference on Machine Learning, ICML'10, Omnipress, Madison, WI, USA, 2010, pp. 807–814.
- [16] K. He, X. Zhang, S. Ren, J. Sun, Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification, in: 2015 IEEE International Conference on Computer Vision (ICCV), IEEE, Santiago, Chile, 2015, pp. 1026–1034.

- doi:10.1109/ICCV.2015.123.  
URL <http://ieeexplore.ieee.org/document/7410480/>
- [17] D.-A. Clevert, T. Unterthiner, S. Hochreiter, Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs) (Feb. 2016). doi:10.48550/arXiv.1511.07289.  
URL <http://arxiv.org/abs/1511.07289>
  - [18] P. Ramachandran, B. Zoph, Q. V. Le, Searching for Activation Functions (Oct. 2017). doi:10.48550/arXiv.1710.05941.  
URL <http://arxiv.org/abs/1710.05941>
  - [19] D. Hendrycks, K. Gimpel, Gaussian Error Linear Units (GELUs) (Jun. 2023). doi:10.48550/arXiv.1606.08415.  
URL <http://arxiv.org/abs/1606.08415>
  - [20] D. Misra, Mish: A Self Regularized Non-Monotonic Activation Function, in: Proceedings of the British Machine Vision Conference 2020, British Machine Vision Association, Online, 2020, p. 191. doi:10.5244/C.34.191.  
URL [https://www.bmva.org/bmvc/2020/conference/papers/paper\\_0928.html](https://www.bmva.org/bmvc/2020/conference/papers/paper_0928.html)
  - [21] C. Dugas, Y. Bengio, F. Bélisle, C. Nadeau, R. Garcia, Incorporating Second-Order Functional Knowledge for Better Option Pricing, in: Advances in Neural Information Processing Systems, Vol. 13, MIT Press, 2000.  
URL [https://papers.nips.cc/paper\\_files/paper/2000/hash/44968aece94f667e4095002d140b5896-Abstract.html](https://papers.nips.cc/paper_files/paper/2000/hash/44968aece94f667e4095002d140b5896-Abstract.html)
  - [22] J. Tompson, R. Goroshin, A. Jain, Y. LeCun, C. Bregler, Efficient Object Localization Using Convolutional Networks (Jun. 2015). doi:10.48550/arXiv.1411.4280.  
URL <http://arxiv.org/abs/1411.4280>
  - [23] V. Dumoulin, F. Visin, A guide to convolution arithmetic for deep learning (Jan. 2018). doi:10.48550/arXiv.1603.07285.  
URL <http://arxiv.org/abs/1603.07285>
  - [24] M. D. Zeiler, D. Krishnan, G. W. Taylor, R. Fergus, Deconvolutional networks, in: 2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, IEEE, San Francisco, CA, USA, 2010, pp. 2528–2535. doi:10.1109/CVPR.2010.5539957.  
URL <http://ieeexplore.ieee.org/document/5539957/>
  - [25] T. Zhang, Y. Yi, Y. Xu, Z. X. Chen, Y. Zhang, W. E, Z.-Q. J. Xu, A multi-scale sampling method for accurate and robust deep neural network to predict combustion chemical kinetics, Combustion and Flame 245 (2022) 112319. doi:10.1016/j.combustflame.2022.112319.  
URL <https://linkinghub.elsevier.com/retrieve/pii/S0010218022003340>
  - [26] H. G. Weller, G. Tabor, H. Jasak, C. Fureby, A tensorial approach to computational continuum mechanics using object-oriented techniques, Computers in Physics 12 (6) (1998) 620–631. doi:10.1063/1.168744.  
URL <https://pubs.aip.org/cip/article/12/6/620/510187/A-tensorial-approach-to-computational-continuum>

- [27] J. Sun, Y. Wang, B. Tian, Z. Chen, detonationFoam: An open-source solver for simulation of gaseous detonation based on OpenFOAM, *Computer Physics Communications* 292 (2023) 108859. doi:10.1016/j.cpc.2023.108859.  
URL <https://linkinghub.elsevier.com/retrieve/pii/S0010465523002047>
- [28] M. Salim, J. Latendresse, S. Khatoonabadi, E. Shihab, Tokenomics: Quantifying Where Tokens Are Used in Agentic Software Engineering, *arXiv:2601.14470 [cs.SE]* (Jan. 2026). doi:10.48550/arXiv.2601.14470.  
URL <http://arxiv.org/abs/2601.14470>
- [29] T. Han, Z. Wang, C. Fang, S. Zhao, S. Ma, Z. Chen, Token-Budget-Aware LLM Reasoning, in: W. Che, J. Nabende, E. Shutova, M. T. Pilehvar (Eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, Association for Computational Linguistics, Vienna, Austria, 2025, pp. 24842–24855. doi:10.18653/v1/2025.findings-acl.1274.  
URL <https://aclanthology.org/2025.findings-acl.1274/>
- [30] D. Grattarola, C. Alippi, Graph Neural Networks in TensorFlow and Keras with Spektral (Jun. 2020). doi:10.48550/arXiv.2006.12138.  
URL <http://arxiv.org/abs/2006.12138>
- [31] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, G. E. Dahl, Neural Message Passing for Quantum Chemistry, in: *Proceedings of the 34th International Conference on Machine Learning*, PMLR, 2017, pp. 1263–1272.  
URL <https://proceedings.mlr.press/v70/gilmer17a.html>
- [32] K. K. Thekumparampil, C. Wang, S. Oh, L.-J. Li, Attention-based Graph Neural Network for Semi-supervised Learning (Mar. 2018). doi:10.48550/arXiv.1803.03735.  
URL <http://arxiv.org/abs/1803.03735>
- [33] G. Guennebaud, B. Jacob, others, *Eigen* (2010).  
URL <https://libeigen.gitlab.io>